AI POWERED RESEARCH GAP IDENTIFIER AND TREND ANALYZER

INDIVIDUAL THESIS

K.L. Widanagama

(IT22629562)

Individual component: A Hybrid Graph Neural Network and KEYBERT-based Recommendation and Collaboration Network for Research Supervisor Selection

Group ID: R26-SE-023

Report submitted in partial fulfilment of the requirements for the

B.Sc. (Hons) in Information Technology

Specializing in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Kusalya Laksarani Widanagama

Student ID: IT22629562

Signature: _____  _____ Date: 00.10. 2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Jeewaka Perera		00.10. 2026
Co-supervisor: Poojani Gunathilake		00.10. 2026

ABSTRACT

Selecting suitable research supervisors is a critical stage in final-year research projects because supervisor expertise, availability, workload, and collaboration fit directly influence project direction, student support, and research quality. In many undergraduate research environments, supervisor selection still depends on manual searching, informal recommendations, static profile browsing, or first-come request handling. These approaches create several problems: students may struggle to identify supervisors whose expertise aligns with their research idea, supervisors may receive repeated requests outside their capacity, and coordinators may lack a transparent mechanism for monitoring allocation fairness.

This research introduces the Recommendation and Collaboration Network, an AI-assisted component designed to recommend suitable supervisors and support the end-to-end supervisor request workflow. The component combines natural language processing, graph-based recommendation, and dynamic workload awareness. Student research titles, abstracts, and keywords are processed using a KeyBERT-based keyword extraction layer to identify topic signals. These signals are compared with supervisor research profiles and embedded into a heterogeneous academic graph consisting of supervisors, research ideas, keywords, and collaboration relationships. A GraphSAGE-based graph neural network generates graph relationship scores, while the system also computes workload feasibility from active primary and co-supervision assignments.

The final recommendation score integrates topic similarity, graph relationship strength, and workload feasibility using a weighted scoring formula. The highest weight is assigned to topic similarity to prioritize academic relevance, while the graph score captures broader research-network relationships and workload feasibility prevents overloading supervisors. The component further implements a request-management workflow where students may send requests for primary supervisor or co-supervisor roles. When a supervisor accepts a request, the research group is directly assigned and competing pending requests for the same group and role are automatically cancelled. Availability notes allow supervisors to communicate short-term constraints without distorting the ranking algorithm.

Keywords: Supervisor Recommendation, KeyBERT, GraphSAGE, Graph Neural Network,GNN, Workload-aware Recommendation, Research Collaboration, Academic Recommender System

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all individuals who contributed their time, guidance, and support throughout this research project.

First and foremost, I am deeply grateful to my supervisor and co-supervisor for their continuous guidance, constructive feedback, and encouragement during the development of this research component. Their advice helped refine the technical direction, evaluation strategy, and academic presentation of this work.

I also extend my appreciation to my group members for their collaboration in developing the system. The integration of the recommendation and collaboration network with the larger system depended on consistent teamwork, shared architectural decisions, and coordinated testing across multiple modules.

Finally, I wish to thank my family and friends for their patience, support, and motivation throughout the research and development process.

Table of Contents

DECLARATION 2
ABSTRACT 3
ACKNOWLEDGEMENT 4
LIST OF TABLES 8
LIST OF FIGURES 9
LIST OF ABBREVIATIONS 10
1. INTRODUCTION 11
1.1 Overall project context 11
1.2 Individual component context 11
1.3 Problem statement 11
1.4 Research Gap 11
1.5 Aim and objectives 13
1.6 Research Problems 13
1.7 Scope and Limitations 14
1.8 Individual contribution 14
1.9 Thesis structure 14
2. LITERATURE REVIEW AND RELATED WORK 15
2.1 Review method and scope 15
2.2 Domain and theoretical foundation 15
2.3 Related research 16
2.4 Existing systems, products and techniques 17
2.5 Gap synthesis and conceptual framework 18
3. RESEARCH METHODOLOGY 20
3.1 Research Design 20
3.2 Data, participants or study materials 21
3.3 Experimental or study procedure 21
3.4 Measures and evaluation criteria 22
3.5 Data analysis 22
3.6 Ethics, privacy, security and responsible AI 23
3.7 Reproducibility and research artefacts 23
4. PRODUCT AND SOLUTION DESIGN 24
4.2 Component architecture within the complete solution 25

4.3 Design alternatives and decisions 26
4.4 Detailed design 27
4.5 Standards and professional practice 28
5. IMPLEMENTATION, INTEGRATION AND TESTING 29
5.1 Development environment and technology stack 29
5.2 Component implementation 30
5.3 Integration with other components 31
5.4 Verification and software testing 31
5.5 Deployment and operation 33
6. RESULTS AND VALIDATION 33
6.1 Research results 33
6.2 Product and Technical Results 34
6.3 User or Stakeholder Validation 35
6.4 Integration Results 35
6.5 Summary of Findings 36
7. Discussion 36
7.1 Interpretation of findings 36
7.2 Comparison with literature baselines and existing solutions 36
7.3 Research contribution 38
7.4 Product contribution 38
7.5 Limitations and threats to validity 38
7.6 Implications 39
8. Commercialization and sustainability 39
8.1 Value proposition and target users 39
8.2 Market and competitive position 39
8.3 Business or sustainability model 40
8.4 Scalability, intellectual property and compliance 40
8.5 Contribution to the Sustainable Development Goals 40
9. Conclusion and future directions 41
9.1 Conclusion 41
9.2 Achievements of objectives 41
9.3 Individual research and product contribution 42
9.4 Future directions 42

References 43
GLOSSARY 45
APPENDICES 46
Appendix A Research instruments or extended results 46
Test case 1 – results output 46
Test case 2 – results from backend similar to the frontend 47
Test 3 – duplication prevention 53
Appendix B Academic integrity and AI use disclosure 53

LIST OF TABLES

Table 1. Comparison of existing supervisor selection approaches with the proposed component 12
Table 2. functional and none functional requirements 25
Table 3. design alternatives 27
Table 4. Technology stack 29
Table 5. Test areas with scenarios 32
Table 6. Test results 34
Table 7. Frontend and backend results 52

LIST OF FIGURES

Figure 1. Recommendation and collaboration flow....26
Figure 2. Topic and supervisor matching flow....28
Figure 3. Supervisor Request flow....33
Figure 4. Results input....46
Figure 5. Results output....46
Figure 6. results from frontend 1....47
Figure 7. Results from frontend 2....47
Figure 8. Results from backend 1....48
Figure 9. Results from backend 2....49
Figure 10. Results from backend 3....50
Figure 11. Results from backend 4....51
Figure 12. Results from backend 5....52
Figure 13. Results from backend 6....52
Figure 14. Duplication Prevention....53

LIST OF ABBREVIATIONS

Abbreviation	**Meaning**
API	Application Programming Interface
DB	Database
GNN	Graph Neural Network
NLP	Natural Language Processing
UI	User Interface

1. INTRODUCTION

1.1 Overall project context

The overall project is an academic research-support software solution designed to assist students, supervisors, and academic administrators during the research project lifecycle. Within such a system, several functions are required, including research idea formulation, supervisor discovery, project administration, progress tracking, and communication between stakeholders. This thesis focuses only on the individual component responsible for recommending suitable supervisors and supporting the practical request and allocation workflow.

1.2 Individual component context

This is an AI-assisted supervisor recommendation and collaboration network that supports students in identifying suitable supervisors and co-supervisors for their research projects. The component allows students to enter a research title, abstract, and keywords through the Find Supervisor interface. These inputs are then processed using KeyBERT-based keyword extraction and topic similarity techniques to identify the main research focus and compare it with supervisor profiles, research interests, and previous supervision areas.

Beyond simple keyword matching, the component also uses graph-based relationship modelling to consider connections between supervisors, research ideas, keywords, and collaboration links. It combines topic relevance, GraphSAGE and GNN-based relationship scoring, and dynamic workload feasibility to produce ranked supervisor recommendation cards. The component also includes a role-based request workflow, allowing students to request a primary supervisor or co-supervisor, while supervisors can review, accept, or reject requests. This connects the recommendation process with the practical academic workflow required for supervisor assignment.

1.3 Problem statement

Students often find it difficult to identify suitable supervisors because research interests are distributed across staff profiles, publications, prior topics, and informal knowledge. Manual selection can depend on coordinator familiarity or peer recommendations, while static staff-profile search requires students to interpret expertise without system support. At the same time, recommending only the most relevant supervisor is insufficient because the selected academic staff member may already be overloaded or unavailable for a particular role. Therefore, the problem addressed in this thesis is the lack of an integrated supervisor recommendation workflow that combines topic relevance, academic relationships, workload feasibility, and supervisor acceptance.

1.4 Research Gap

Existing supervisor selection approaches in many educational settings can be grouped into three broad categories: manual allocation, static profile search, and basic keyword-based matching. Manual allocation depends heavily on coordinator knowledge and is difficult to scale. Static profile search gives students access to staff information but still requires students

to interpret expertise manually. Basic keyword matching can improve retrieval but often fails to capture indirect academic relationships and collaboration patterns. [1], [2], [3], [4]

A major gap is the limited use of graph-based academic relationship modelling in practical supervisor selection workflows. A supervisor may be suitable for a topic not only because the profile contains the same keywords but also because the supervisor has previously supervised related topics, collaborates with related researchers, or is connected to a cluster of research ideas in the same area. Traditional keyword matching cannot fully exploit these relationships. [5], [6], [7], [8]

Another gap is workload awareness. A recommendation system that only ranks expertise may repeatedly recommend the same highly relevant supervisors even when they are already overloaded. This creates a mismatch between recommendation quality and practical feasibility. In a real academic setting, the system must consider active primary supervision load, co-supervision load, and capacity thresholds while still preserving relevance. [9], [6], [7]

A further gap is the separation between recommendation and workflow execution. Many recommender systems stop at producing a ranked list. In practice, supervisor allocation requires request creation, supervisor review, acceptance or rejection, automatic assignment, prevention of duplicate or conflicting requests, and visibility of short-term availability constraints. The proposed component addresses this by integrating the recommendation layer with a role-based request and assignment workflow. [10], [9], [4], [11]

Table 1. Comparison of existing supervisor selection approaches with the proposed component

Approach	Strength	Limitation	How the proposed component improves it
Manual coordinator allocation	Uses human knowledge and institutional context	Difficult to scale; may be inconsistent; limited transparency	Provides ranked evidence to support decisions while preserving final human control
Static supervisor profile browsing	Simple and easy to understand	Students must manually interpret expertise and availability	Automatically compares student topics with supervisor profiles
Keyword-only matching	Faster than manual search	Misses graph relationships and collaboration context	Adds GraphSAGE-based relationship scoring
Expertise-only recommendation	Can identify academically relevant supervisors	May overload popular supervisors	Adds dynamic workload feasibility and availability notes

1.5 Aim and objectives

The main objective of this research is to design and implement an AI-assisted Supervisor Recommendation and Collaboration Network that helps students identify suitable supervisors and co-supervisors by combining KeyBERT-based topic understanding, graph-based relationship modelling, workload-aware ranking, and a role-based request and assignment workflow.

The specific objectives are as follows:

1. To design and implement a supervisor recommendation pipeline that allows students to submit research titles, abstracts, and keywords through the Find Supervisor interface, and prepares these inputs for topic analysis and supervisor matching.
2. To implement KeyBERT-based keyword extraction and text similarity matching to identify the main research areas of student ideas and compare them with supervisor profiles, research interests, and previous supervision areas.
3. To design a heterogeneous academic graph that represents supervisors, research ideas, keywords, expertise areas, and collaboration relationships as connected entities within the recommendation process.
4. To apply GraphSAGE and GNN-based scoring to capture relationship-aware supervisor relevance by considering indirect academic links between supervisors, research topics, keywords, and related research areas.
5. To integrate dynamic workload feasibility into the supervisor ranking process by considering active primary supervision and co-supervision assignments, so that recommendations reflect both expertise and practical supervision capacity.
6. To develop a role-based supervisor request workflow that supports primary supervisor and co-supervisor requests, supervisor review, acceptance, rejection, automatic role assignment, and cancellation of conflicting pending requests.
7. To provide supervisor availability and workload information in the user interface so that students can make informed decisions when selecting and requesting supervisors.
8. To evaluate the component using realistic test cases by reviewing recommendation relevance and validating workflow correctness through request creation, acceptance, assignment, cancellation, and role-based restriction scenarios.

1.6 Research Problems

Three research questions guide this component. First, can a hybrid NLP and graph-based recommendation method produce academically relevant supervisor suggestions for student research topics? Second, can workload-aware ranking improve the practical feasibility of supervisor recommendations without completely removing academically relevant supervisors from consideration? Third, can the recommendation system be integrated with a role-based request workflow that directly supports final supervisor and co-supervisor assignment?

These questions reflect both algorithmic and system-level concerns. The component is not evaluated only as a model but also as a deployable workflow. Therefore, the research

examines recommendation relevance, database consistency, user interface behaviour, and acceptance-driven assignment logic.

1.7 Scope and Limitations

The scope of this research is limited to the Recommendation and Collaboration Network component. The component covers supervisor recommendation, collaboration-aware ranking, dynamic workload computation, supervisor availability notes, and the request/assignment workflow. It does not cover the full research gap identification module, the complete collaborative workspace, or institution-wide administrative allocation policies outside the implemented platform.

The recommendation model depends on the quality and completeness of available supervisor profile data, previous research idea records, and group assignment data. If profiles are incomplete or outdated, the system may not fully reflect actual expertise. The evaluation uses manual relevance validation over a defined set of research topics rather than a long-term field deployment with real acceptance outcomes across multiple academic years. These limitations are addressed as future work.

1.8 Individual contribution

The individual contribution includes the design and development of the end-to-end supervisor recommendation and assignment process. This includes allowing students to submit their research ideas, generating suitable supervisor suggestions, presenting recommendation details clearly, showing workload and availability information, and enabling students to send role-specific requests for primary supervisor or co-supervisor roles.

The contribution also includes the workflow required after a recommendation is made, such as supervisor review, acceptance or rejection of requests, automatic assignment of accepted supervisors, and cancellation of conflicting pending requests. Therefore, the contribution is not limited to the recommendation model itself, but covers the complete process of turning a recommendation into a practical supervisor allocation outcome.

1.9 Thesis structure

2. LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The literature review focused on existing work related to academic recommender systems, supervisor recommendation, academic collaborator recommendation, KEYBERT-based academic matching, multi-criteria decision support, and workload-aware allocation. These areas were selected because they directly relate to the problem of matching students with suitable academic supervisors or address similar academic recommendation tasks.

The reviewed studies were used to understand how existing systems perform supervisor or academic expert matching, what techniques they commonly use, and what limitations remain. This helped identify the gap addressed by this component, particularly the need to combine recommendation, workload awareness, and supervisor request handling into one practical workflow.

2.2 Domain and theoretical foundation

Recommender systems are commonly used to reduce information overload by helping users identify items that are likely to be relevant or useful to them. Traditional recommender systems are generally grouped into content-based filtering**,** collaborative filtering**, and** hybrid approaches [12]. Content-based filtering recommends items by comparing the features of an item with the user's interests, while collaborative filtering uses patterns from previous user-item interactions. Hybrid approaches combine multiple signals to overcome the weaknesses of a single method. In academic contexts, recommender systems can be applied to papers, courses, collaborators, examiners, supervisors, or research topics. However, supervisor recommendation is different from many commercial recommendation tasks because the recommended "item" is a human academic expert with limited supervision capacity, institutional responsibilities, and role-based constraints.

Natural language processing is important in this component because student research ideas are usually provided as short titles, abstracts, and keyword lists. These inputs are unstructured and may vary in quality, detail, and wording. In this component, KeyBERT-based keyword extraction is used as the practical NLP method to identify important terms and keyphrases from the student's research input. These extracted terms help identify the central research area of the proposed topic and support topic matching between student ideas and supervisor expertise. This means the system does not depend only on manually entered keywords, but also extracts meaningful terms from the title and abstract to improve the matching process.

However, topic matching alone is not sufficient for supervisor recommendation. A supervisor may be suitable even when the exact same words do not appear in both the student's topic and the supervisor profile. Therefore, the component also uses graph-based academic modelling to represent relationships between supervisors, research ideas, keywords, expertise areas, and collaboration links. This allows the system to consider indirect academic relationships, such as previous supervision of related topics or connections to similar research areas.

The use of GraphSAGE with GNN-based scoring is motivated by this need to capture relationship-aware relevance. Instead of treating supervisor recommendation as only a text

similarity problem, the graph-based approach allows the system to analyse how supervisors, topics, and keywords are connected within the academic environment. This helps the recommendation process move beyond surface-level keyword overlap and produce suggestions that better reflect the broader academic context. Together, KeyBERT-based topic understanding, text similarity, graph-based scoring, and workload feasibility form the foundation of the proposed recommendation approach.

2.3 Related research

Rismanto et al. [1], Falah and Suryawan [2], and Wijanto et al. [3] show that content-based recommendation methods can support supervisor selection by comparing student research proposals with supervisor expertise, research interests, or academic publications. These studies mainly rely on techniques such as TF-IDF, vector space modelling, and cosine similarity to calculate how closely a student's topic matches a supervisor's profile. Their work demonstrates that text-based similarity is useful for reducing manual searching and helping students identify academically relevant supervisors.

Fitri et al. [10] extend this direction by using a hybrid filtering approach that combines content-based and collaborative recommendation signals. This shows that supervisor recommendation can benefit from using more than one matching method instead of relying only on direct keyword similarity. Maulani et al. [9] also highlight an important practical issue in supervisor allocation: the system should not only consider expertise alignment, but also workload distribution. This is relevant because a highly suitable supervisor may not be practically available if their supervision load is already high.

Other studies focus on related aspects of the supervisor selection problem. Zemaityte and Terzic [4] present a supervisor recommendation tool for computer science projects, showing how such systems can guide students during the supervisor selection stage. Madeira et al. [5] use academic genealogy to recommend supervisors based on supervision history, which highlights the value of previous academic relationships. Hasan and Schwartz [6], together with Hasan [7], propose multi-criteria decision-support approaches that consider factors such as publications, research grants, and collaboration networks. These studies show that supervisor selection is not only a topic-matching problem, but a decision-making process involving several academic indicators.

Liu et al. [8] contribute to this area by focusing on context-aware academic collaborator recommendation, which is useful for understanding how academic relationships and research context can influence recommendation quality. Dasri et al. [11] explore two-way thesis supervisor recommendation, where the preferences of both students and supervisors are considered. Overall, these studies provide useful foundations for the proposed component. However, they also show that there is still a need for a more integrated approach that combines supervisor recommendation, workload feasibility, academic relationship modelling, and role-based request handling within one practical workflow.

2.4 Existing systems, products and techniques

The reviewed studies show that existing supervisor recommendation approaches address different parts of the supervisor selection problem. Rismanto et al. [1] use TF-IDF and cosine similarity to compare student topics with supervisor expertise, while Falah and Suryawan [2] apply a content-based filtering approach for supervisor recommendation. Similarly, Wijanto et al. [3] use information retrieval techniques to recommend supervisors based on academic publication similarity. These studies demonstrate that text-based topic matching is useful for identifying relevant supervisors, but they mainly focus on direct similarity between student input and supervisor-related information. They do not fully consider academic relationships, supervisor workload, or the request and assignment process that follows a recommendation.

Fitri et al. [10] extend content-based recommendation by combining content-based filtering and collaborative filtering in a hybrid approach. This improves the recommendation process by considering more than one signal, but the approach still depends heavily on fixed weighting and relatively shallow text similarity. Maulani et al. [9] address supervisor allocation from a different direction by considering expertise alignment and workload distribution using a C4.5-based allocation approach. This is important because supervisor selection is not only about topic relevance, but also about whether a supervisor has sufficient capacity to take on new students. However, this type of allocation approach provides limited support for graph-based academic relationship modelling.

Other studies focus on relationship-based and multi-criteria supervisor recommendation. Madeira et al. [5] use academic genealogy to recommend supervisors based on previous supervision relationships, showing the value of supervision history in academic recommendation. Hasan and Schwartz [6] apply a multi-criteria recommendation approach that considers factors such as publications, grants, and collaboration-related criteria. These studies show that supervisor suitability can depend on more than direct topic similarity. However, they are mainly decision-support approaches and do not fully integrate the operational workflow needed for supervisor requests, acceptance, assignment, and conflict handling.

In addition to research prototypes, several real-time supervisor discovery systems are already used by universities and postgraduate students. For example, the University of Auckland provides a “Find a Supervisor” facility through its Discovery Profiles database, where students can search for supervisors and research opportunities using topic keywords [13]. The University of Galway also provides a “Find a Supervisor /PhD Project” portal that allows students to search potential supervisors or projects and filter results by research unit, time period, concepts, Sustainable Development Goals, and whether staff are currently accepting PhD students [14]. Similarly, Monash University’s Supervisor Connect allows students to browse or search research projects and supervisors, read about research strengths, and initiate an application from the same platform [15]. External platforms such as FindAPhD also provide supervisor search functionality, allowing prospective PhD students to search for supervisors based on research interests [16].

These systems show that real-time supervisor discovery is already an important feature in higher education environments. However, most of these systems mainly operate as searchable

directories, project discovery portals, or supervisor browsing tools. They help students find academic staff by research area, keyword, project, or availability, but they do not fully demonstrate an integrated recommendation and allocation workflow. In particular, they generally do not combine student research idea processing, AI-assisted topic extraction, graph-based relationship scoring, dynamic workload feasibility, role-specific supervisor and co-supervisor requests, supervisor acceptance or rejection, automatic assignment, and cancellation of conflicting requests in one workflow. Therefore, the proposed component is positioned not only as a supervisor discovery tool, but as a recommendation and assignment support component that connects supervisor matching with the practical steps required for academic allocation.

Overall, the reviewed systems provide useful foundations for supervisor recommendation, including topic matching, hybrid recommendation, workload consideration, academic genealogy, and multi-criteria decision support. However, they usually address these aspects separately. The proposed component addresses this gap by combining KeyBERT-based topic understanding, graph-based relationship scoring, workload feasibility, and a role-based request and assignment workflow. This allows the component to support both the recommendation decision and the practical process of assigning supervisors and co-supervisors.

2.5 Gap synthesis and conceptual framework

The reviewed literature shows that existing supervisor recommendation and academic recommender systems have addressed several important parts of the supervisor selection problem. Some studies focus mainly on topic matching, where student research proposals are compared with supervisor profiles, publications, or research interests. Other studies extend this through hybrid recommendation**,** multi-criteria decision support**,** academic genealogy, or workload-based allocation. These approaches show that supervisor selection cannot be treated as a simple search task. It involves matching the student's research direction with suitable academic expertise while also considering supervision history, collaboration patterns, and practical allocation constraints.

However, the main limitation identified from the literature is that these functions are often handled separately. A system may support topic similarity but ignore workload. Another system may consider workload but provide limited support for academic relationship modelling. Similarly, some systems recommend supervisors but do not support the actual request, acceptance, and assignment process that follows. This creates a gap between producing a recommendation and completing a practical supervisor allocation workflow. In real academic settings, a suitable supervisor must not only be relevant to the topic, but also available, within workload limits, and willing to accept the student or group.

Based on this gap, the conceptual framework of this component is developed around four connected layers. The first layer is topic understanding, where the student's research title, abstract, and keywords are processed to identify the main research area. The second layer is graph-based academic relationship scoring, which considers relationships between supervisors, research ideas, keywords, expertise areas, and collaboration links. The third layer

is workload feasibility, which checks whether a supervisor's current supervision load allows them to take on additional students. The fourth layer is the role-based request workflow, where students can send requests, supervisors can accept or reject them, and the system can update the final supervisor or co-supervisor assignment.

Therefore, the conceptual framework treats supervisor selection as both a recommendation problem and an academic workflow problem. The recommendation layer helps identify suitable supervisors, while the workflow layer ensures that the recommendation can be acted on in a controlled and practical way. This layered design supports a more realistic supervisor allocation process by combining relevance, academic relationships, workload awareness, and human decision-making.

3. RESEARCH METHODOLOGY

3.1 Research Design

This research follows an applied design-and-development methodology, as the main objective is to design, implement, and evaluate a functional software component for supervisor recommendation and assignment support. The research is not limited to theoretical model development, but focuses on building a working component that can process student research details, generate supervisor recommendations, support request handling, and manage supervisor assignment decisions. Therefore, the methodology includes system requirement analysis, recommendation logic design, database integration, user interface development, backend service integration, and validation through realistic functional test scenarios.

The component is designed using a multi-layer architecture. The frontend layer provides the student-facing and supervisor-facing interfaces, including the Find Supervisor page, recommendation cards, request buttons, workload indicators, and supervisor availability notes. The backend layer handles authentication, database communication, request creation, supervisor decision handling, assignment updates, and cancellation of conflicting requests. A separate recommendation service processes the student's research title, abstract, and keywords, applies KeyBERT-based topic extraction, compares the topic with supervisor expertise, and combines topic similarity with graph-based scoring and workload feasibility. The database layer stores supervisor profiles, research interests, availability notes, supervision requests, role assignments, workload-related records, and recommendation-related data.

The research was developed iteratively. The initial stage focused on building the core recommendation pipeline, where student research input is accepted, processed, and converted into ranked supervisor suggestions. The next stage introduced graph-based relationship scoring to improve the recommendation process beyond direct keyword matching. After that, dynamic workload feasibility was integrated so that supervisor recommendations reflect both relevance and practical capacity. The later stages focused on operational workflow features, including role-specific supervisor requests, supervisor acceptance and rejection, automatic supervisor or co-supervisor assignment, availability note handling, and cancellation of pending conflicting requests.

Each development iteration was validated against both technical requirements and workflow requirements. Technical validation focused on whether the recommendation pipeline, database operations, API communication, and interface behaviour worked correctly. Workflow validation focused on whether the system supported the real academic process, such as allowing a student to request a supervisor, allowing the supervisor to review and respond, updating the assigned role after acceptance, and preventing duplicate or conflicting assignments. This design-and-development approach was suitable because the component needed to demonstrate both recommendation capability and practical usability within an academic supervision workflow.

3.2 Data, participants or study materials

The study materials used for this component consisted of academic and project-related information required to support supervisor recommendation and assignment. These included supervisor profile details, research interests, areas of expertise, previously supervised research topics, keyword information, and available relationship or collaboration data. These materials were used to represent the academic context needed for matching student research ideas with suitable supervisors and co-supervisors.

Controlled student research ideas were also used during development and testing. These research ideas included sample titles, abstracts, and keywords that reflected realistic student project proposals. They were used to test whether the component could process research input correctly, identify relevant topic areas, generate suitable supervisor recommendations, and support the request workflow from the student and supervisor sides.

The study also considered supervision-related operational data, such as existing group assignments, active supervision responsibilities, co-supervision involvement, request status, and supervisor availability information. These materials were necessary to test workload feasibility and role-based assignment behaviour. Any future deployment should use institutionally approved academic data, with proper access control, privacy protection, and role-based permissions.

3.3 Experimental or study procedure

The procedure began with the preparation of controlled student research ideas that represented realistic project proposals. Each research idea included a title, abstract, and keyword list. These inputs were entered through the Find Supervisor interface to test how the component handles student-submitted research information and converts it into a form suitable for supervisor matching.

After submission, the input was processed by the recommendation service. Important keywords and keyphrases were extracted from the student's research idea and compared with supervisor profile information, research interests, and previous supervision areas. This stage was used to identify supervisors whose expertise was relevant to the submitted topic. In addition to direct topic matching, the procedure also used graph-based relationship scoring to consider links between supervisors, research ideas, keywords, and collaboration-related information.

The next stage involved workload feasibility checking. The system considered active primary supervision and co-supervision responsibilities to determine whether a supervisor was practically suitable for another assignment. The final recommendation cards were then reviewed to confirm that they displayed relevant supervisor details, ranking information, workload indicators, availability notes, and role-specific request options.

Finally, the operational workflow was tested using the generated recommendations. This included creating supervisor and co-supervisor requests, reviewing requests from the supervisor side, accepting or rejecting requests, automatically assigning accepted supervisors to the correct role, and cancelling conflicting pending requests. This procedure helped

validate both the recommendation process and the practical workflow required for supervisor allocation.

3.4 Measures and evaluation criteria

Evaluation focused on checking whether the component behaved correctly and produced useful supervisor recommendations under realistic usage scenarios. Since the component includes both a recommendation layer and an operational workflow, the evaluation was not limited to the ranking output alone. It also considered whether students could submit research ideas, receive relevant supervisor suggestions, view meaningful explanation details, and continue the process by sending supervisor or co-supervisor requests.

The recommendation output was reviewed by checking whether the suggested supervisors were relevant to the submitted research title, abstract, and keywords. This included observing whether the system could identify supervisors with matching or closely related research interests, and whether the ranking reflected the combined effect of topic relevance, graph-based relationship scoring, and workload feasibility. The evaluation also checked whether the recommendation cards displayed the required information clearly, including supervisor details, relevance explanations, workload indicators, availability notes, and role-specific request buttons.

The workflow evaluation focused on the correctness of the supervisor request and assignment process. This included testing whether students could create requests for the correct role, whether duplicate or invalid requests were restricted, and whether supervisors could accept or reject requests from their side. When a request was accepted, the system was checked to confirm that the correct supervisor or co-supervisor role was updated automatically. The system was also tested to ensure that conflicting pending requests for the same role were cancelled or disabled after an assignment was made.

Workload and availability behaviour was also reviewed as part of the evaluation. The workload labels were checked against active primary supervision and co-supervision responsibilities to confirm that the displayed capacity status was consistent with the supervisor's current load. Availability notes were checked to ensure that supervisor-provided notes were visible to students and updated correctly when changed.

Finally, integration testing was carried out across the frontend, backend, recommendation service, and database layers. This ensured that student input, recommendation generation, request creation, supervisor decisions, assignment updates, and interface refreshes worked together as one complete workflow.

3.5 Data analysis

Data analysis was carried out by reviewing how the component handled each stage of the recommendation and assignment workflow. As part of the development and testing process, a synthetic dataset was created to represent realistic academic supervision scenarios. This dataset included sample supervisor profiles, research interests, previous topic areas, keyword information, collaboration relationships, student research ideas, supervision workload details,

and request states. The use of synthetic data allowed the component to be tested safely without relying on sensitive institutional records or real student-supervisor allocation data.

The submitted research topics were compared with the extracted keywords and keyphrases to check whether the system correctly identified the main research area of each student proposal. The recommendation output was then examined to determine whether the suggested supervisors were logically related to the submitted topic and whether the ranking reflected topic relevance, graph-based relationship scoring, and workload feasibility. The synthetic dataset was useful in this stage because it allowed different types of research topics, supervisor expertise areas, workload levels, and collaboration patterns to be tested in a controlled manner.

The analysis also considered the operational behaviour of the component. Functional logs, stored records, and user interface states were inspected to verify that each workflow step produced the expected result. This included checking whether requests were created correctly, whether supervisors could accept or reject requests, whether accepted requests updated the assigned supervisor or co-supervisor role, and whether the interface reflected these changes correctly for both students and supervisors.

Special attention was given to edge cases because these situations are important in a real supervisor allocation process. The analysis included scenarios such as duplicate requests, requests sent after a role had already been assigned, pending requests for the same role, unavailable or overloaded supervisors, and workload changes after supervisor acceptance. These cases were reviewed to confirm that the component handled conflicts correctly, prevented invalid actions, and maintained a consistent assignment state across the frontend, backend, recommendation service, and stored data.

3.6 Ethics, privacy, security and responsible AI

3.7 Reproducibility and research artefacts

The reproducible artefacts of this component include the frontend interfaces, backend API functions, recommendation service modules, database structure updates, configuration files, test scenarios, and validation notes. These artefacts are important because they allow the component to be re-created, tested, and reviewed in a consistent manner. The frontend artefacts include the student-facing supervisor recommendation interface, recommendation cards, workload and availability displays, and supervisor request actions. The backend artefacts include the routes and logic used for request creation, supervisor review, acceptance, rejection, assignment updates, and cancellation of conflicting requests.

The recommendation-related artefacts include the modules used for processing student research input, extracting important keywords, comparing topics, applying graph-based scoring, and calculating workload feasibility. These modules need to be maintained together with their configuration settings so that the recommendation behaviour can be reproduced in the same way during future testing or deployment. The database-related artefacts include the structural updates required to store supervisor information, research-related data, workload details, availability notes, requests, and role assignments.

Reproducibility also depends on maintaining a clear development and execution environment. This includes documenting the required environment variables, service ports, API contracts, dependency versions, database migration steps, and instructions for running the frontend, backend, and recommendation service. The synthetic dataset, test cases, and validation notes also support reproducibility because they provide controlled examples for checking whether the component behaves as expected.

Overall, the component can be reproduced if the implementation artefacts are version-controlled, the configuration is clearly documented, and the testing procedure is preserved. This ensures that another developer or evaluator can set up the same environment, run the recommendation workflow, test supervisor request handling, and verify whether the system produces consistent functional behaviour.

4. PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The component is designed to accept student research details, including the research title, abstract, and keywords, through the student-facing interface. These inputs are required so that the system can understand the proposed research topic and prepare it for supervisor matching. After the input is submitted, the system must extract relevant topic keywords and calculate topic similarity between the student's idea and available supervisor profiles. This is a high-priority functional requirement because the quality of the recommendation depends on how well the system understands and compares research topics.

The component must also support relationship-aware supervisor recommendation using graph-based scoring. This allows the system to consider not only direct topic similarity, but also academic relationships between supervisors, research areas, keywords, previous topics, and collaboration links. In addition, the system must calculate workload feasibility using active supervisor and co-supervisor assignments. This ensures that recommendations are not based only on expertise, but also on whether a supervisor has practical capacity to take on additional students.

Another key functional requirement is the support for role-specific supervisor requests. Students should be able to request an academic staff member as either a primary supervisor or a co-supervisor. The system must allow supervisors to review these requests and either accept or reject them. When a request is accepted, the system must automatically update the correct assigned role and cancel or disable conflicting pending requests for the same role. This is important to avoid duplicate assignments and maintain a clear supervisor allocation state.

The component also has important non-functional requirements. Student and supervisor data must be protected through role-based access, so that users can only view and perform actions relevant to their role. For example, students should manage their own requests, while supervisors should only review requests assigned to them. The system should also provide clear and understandable recommendation feedback, including relevance explanations,

workload indicators, and availability information. This helps students understand why a supervisor is recommended and supports more informed decision-making.

Table 2. functional and none functional requirements

ID	Requirement	Type	Priority
FR1	Accept research title, abstract, and keywords from students.	Functional	High
FR2	Extract topic keywords and compute topic similarity.	Functional	High
FR3	Use graph-based scoring to support relationship-aware recommendations.	Functional	High
FR4	Calculate workload feasibility from active assignments.	Functional	High
FR5	Support role-specific supervisor and co-supervisor requests.	Functional	High
FR6	Automatically assign accepted roles and cancel conflicting pending requests.	Functional	High
NFR1	Protect student and supervisor data through role-based access.	Non-functional	High
NFR2	Provide clear and understandable recommendation feedback.	Non-functional	Medium

4.2 Component architecture within the complete solution

The component is organised as a frontend recommendation interface, backend request and database layer, Python-based recommendation service, and academic database. The frontend collects student input and displays ranked recommendation cards. The backend validates authenticated requests, stores supervision requests, coordinates role assignment, and exposes availability information. The recommendation service performs topic processing, graph scoring, workload calculation, and result formatting.

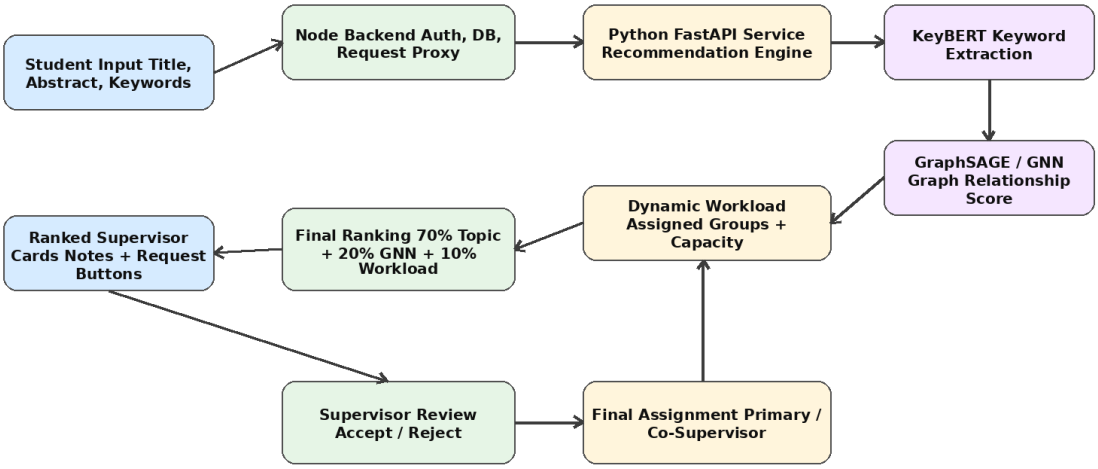


Figure 1. Recommendation and collaboration flow

4.3 Design alternatives and decisions

Several design alternatives were considered during the development of the component. For topic extraction, the initial approach was to depend on manually entered student keywords and simple TF-IDF-based matching. This was useful as an early baseline because it allowed student topics and supervisor profiles to be compared using weighted terms. However, this approach depended heavily on the quality of the keywords entered by students and was limited when students used incomplete, vague, or different wording from supervisor profiles. Therefore, the selected decision was to use KeyBERT-assisted keyword extraction, so that important terms and keyphrases could also be extracted from the research title and abstract.

For the recommendation model, a keyword-search-only approach was first considered because it is simple to implement and easy to understand. However, keyword search alone cannot capture indirect academic relationships, such as supervisors connected to similar past topics, related research areas, or collaboration links. Therefore, the selected design uses a hybrid approach that combines topic similarity with graph-based scoring. This allows the recommendation process to consider both direct textual relevance and wider academic relationships.

For workload handling, one alternative was to display workload information only as an additional detail on the supervisor card. However, this would mean that workload does not influence the recommendation process itself. The selected decision was to include workload as a feasibility layer, so that supervisor recommendations reflect not only expertise but also practical supervision capacity. This makes the component more useful in a real academic environment where a relevant supervisor may still be unsuitable if already overloaded.

For the allocation process, direct student selection was considered as a possible approach. Under that method, a student could choose a supervisor directly from the recommendation list. However, this does not reflect the academic responsibility of supervisors, since a supervisor should have the opportunity to review the topic and decide whether to accept the

role. Therefore, the selected decision was to use a request-based workflow where students send requests and supervisors accept or reject them.

Finally, request conflict handling was also considered during the design. One possible approach was to allow multiple accepted requests to remain active, but this could create duplicate or conflicting role assignments. The selected decision was to automatically cancel or disable conflicting pending requests once a supervisor accepts a role. This helps maintain a clear assignment state and prevents more than one supervisor from being assigned to the same role unintentionally.

Table 3. design alternatives

Design issue	Alternative considered	Selected decision	Rationale
Topic extraction	Manual keywords only	KeyBERT-assisted extraction	Reduces dependence on student keyword quality.
Recommendation model	Keyword search only	Hybrid topic and graph scoring	Captures indirect academic relationships.
Workload handling	Display only	Use workload in feasibility layer	Improves practical usefulness of recommendations.
Allocation process	Student selects directly	Supervisor accepts request	Preserves human approval and academic responsibility.
Request conflicts	Allow multiple active acceptances	Cancel conflicting pending requests	Prevents duplicate role assignment.

4.4 Detailed design

The detailed design of the component consists of six main sub-processes that work together to support supervisor recommendation and assignment. The process begins when a student submits a research idea through the user interface by entering a research title, abstract, and keywords. This input forms the main basis for identifying the research area of the student's proposed project. The interface is designed to collect the information in a structured way so that it can be processed consistently by the recommendation service.

After the research idea is submitted, the text is processed to identify the most important terms and keyphrases. KeyBERT is used to extract meaningful keywords from the title and abstract, reducing the system's dependence on manually entered student keywords alone. This is important because students may use incomplete, general, or inconsistent keywords when describing their research ideas. By extracting keyphrases from the full text, the system can better understand the main topic of the proposal and prepare it for comparison with supervisor expertise.

The next sub-process is topic similarity calculation. The extracted keywords, student-provided keywords, and research text are compared with supervisor profiles, research interests, and previous supervision areas. This produces an initial relevance indication between the student's topic and each potential supervisor. This layer ensures that supervisors with closely related expertise are given higher consideration in the recommendation process.

The graph layer then adds relationship-aware scoring to the recommendation process. In this layer, supervisors, research ideas, keywords, expertise areas, and collaboration links are represented as connected entities. This allows the system to consider indirect academic relationships that may not be captured through text similarity alone. For example, a supervisor may be relevant because they have supervised similar topics before or are connected to related research areas through collaboration or expertise links.

Workload feasibility is then applied to make the recommendations more practical. The system considers active supervision responsibilities, including primary supervision and co-supervision assignments, when assessing whether a supervisor is suitable for another student group. This helps prevent the system from recommending only the most relevant supervisors without considering whether they have the capacity to take on additional supervision work.

Finally, the workflow layer connects the recommendation output with the actual supervisor allocation process. Students can send role-specific requests to recommended academic staff as either a primary supervisor or co-supervisor. Supervisors can review the request and accept or reject it. When a request is accepted, the system updates the relevant assignment and handles conflicting pending requests to prevent duplicate role allocation. This ensures that the component does not stop at recommendation, but supports the complete process from student research input to supervisor assignment.

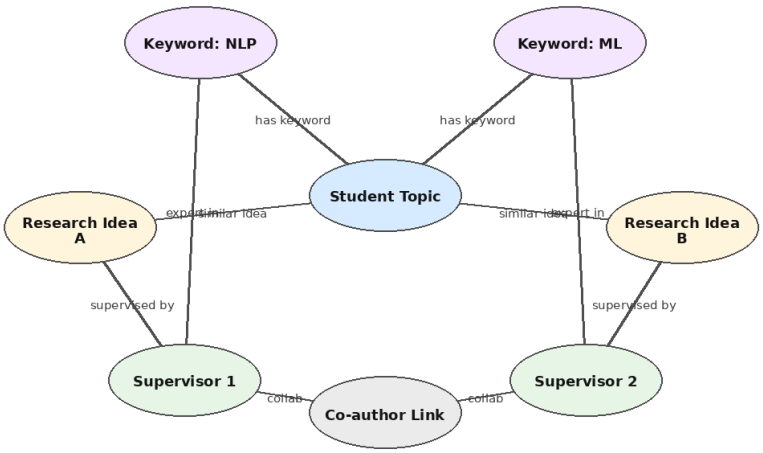


Figure 2. Topic and supervisor matching flow

The final score is designed as a weighted combination of topic similarity, graph-based relevance, and workload feasibility. The exact weights can be configured depending on institutional priorities. The design intentionally keeps supervisor acceptance in the workflow so that recommendation output remains advisory rather than fully automatic.

4.5 Standards and professional practice

The design follows accepted software engineering principles to make the component maintainable, reliable, and suitable for integration within a larger academic management system. The component is divided into clear service boundaries, where the user interface, backend logic, recommendation service, and database each handle separate responsibilities.

This separation of concerns reduces complexity and makes it easier to test, update, or replace individual parts of the system without affecting the full workflow. API-based integration allows communication between the frontend, backend, and recommendation service, while database-backed state management ensures that requests, assignments, workload values, and supervisor availability information are stored consistently.

The component also follows professional practices such as input validation, error handling, and controlled access to role-specific operations. Student inputs must be validated before processing, and request actions must be restricted according to user role so that students and supervisors can only perform permitted actions. From a responsible AI perspective, the system treats recommendations as decision-support outputs rather than final decisions. The interface should explain recommendation factors such as topic relevance, workload condition, and relationship-based relevance, while final assignment remains subject to supervisor review and approval. This helps ensure that the component is technically reliable, ethically appropriate, and aligned with responsible use of AI-assisted academic decision support.

5. IMPLEMENTATION, INTEGRATION AND TESTING

5.1 Development environment and technology stack

The implementation uses a multi-service architecture consisting of a React-based frontend, a Node.js/Express backend, a FastAPI-based recommendation service, and a PostgreSQL-compatible database. The frontend is responsible for user interaction, including student research input, supervisor recommendation cards, workload indicators, availability notes, and request actions. The backend handles authentication, database communication, request creation, supervisor response handling, assignment updates, and conflict cancellation. The recommendation service is separated from the main backend so that recommendation-related processing can be managed independently from the normal workflow operations.

The recommendation service uses KeyBERT to support keyword and keyphrase extraction from student research titles and abstracts. These extracted terms are used for topic matching with supervisor profiles, while the graph-scoring module supports GraphSAGE/GNN-based relevance scoring to consider academic relationships beyond direct keyword similarity. The database stores the persistent state of the component, including supervisor information, research-related data, workload details, availability notes, requests, and assignments. This architecture separates user interaction, workflow management, recommendation computation, and data storage, making the component easier to maintain, test, and extend.

Table 4. Technology stack

Layer	Technology	Purpose
Frontend	React with Vite	Student and supervisor user interfaces
Backend	Node.js and Express	Authentication, API routing, database operations, request

		workflow
Recommendation service	Python FastAPI	NLP, graph scoring, workload-aware ranking
Database	PostgreSQL	Persistent storage for users, groups, supervisors, and requests
NLP	KeyBERT / transformer embeddings	Keyword extraction and semantic topic representation
Graph learning	GraphSAGE / GNN	Relationship-aware supervisor scoring
Data processing	pandas, scikit-learn, NumPy	Data loading, similarity scoring, and evaluation utilities
Frontend API layer	backend_api.js	Centralized REST calls from React components

5.2 Component implementation

The frontend implementation was developed to support the complete student-facing supervisor discovery process. The Find Supervisor interface allows students to enter their research title, abstract, and keywords in a structured form. Once the student submits the details, the frontend sends the input to the backend and displays the returned supervisor recommendations as individual recommendation cards. Each card presents the supervisor's basic details, relevance information, workload status, availability note, and action buttons. The request buttons are role-specific, allowing the student to send a request either for a primary supervisor role or a co-supervisor role. The frontend also updates button states based on the current workflow status, such as pending request, accepted request, rejected request, or already assigned role.

The backend implementation was developed as the main workflow control layer. It receives student actions from the frontend, validates the request, communicates with the database, and ensures that the supervisor assignment process remains consistent. When a student sends a supervisor request, the backend checks whether the same request already exists, whether the requested role is already filled, and whether the action is valid for the current group. It then creates the request and stores the relevant status. When a supervisor accepts or rejects a request, the backend updates the request status and, in the case of acceptance, assigns the supervisor or co-supervisor to the correct role. It also cancels conflicting pending requests for the same role so that duplicate assignments do not occur.

The recommendation service was implemented separately to handle the AI-assisted recommendation logic. It loads supervisor-related information, topic data, keyword information, and relationship data required for recommendation generation. When a student submits a research idea, the service processes the title, abstract, and keywords, extracts important keywords and keyphrases, and compares them with supervisor profiles and research interests. It then combines text-based similarity with graph-based scoring to identify

supervisors who are relevant not only by direct topic match but also through academic relationships such as related topics, expertise areas, and collaboration links.

Workload handling was also integrated into the recommendation process. The service refreshes workload-related data from active supervision and co-supervision assignments and uses this information to calculate practical feasibility. This allows the final recommendation output to reflect both academic relevance and supervision capacity. After computing the topic similarity, graph relevance, and workload feasibility, the recommendation service formats the results into a frontend-ready structure. This allows the user interface to display ranked supervisor cards with explanation details, workload labels, availability information, and request actions in a consistent way.

5.3 Integration with other components

The component is integrated with the wider system through several connected modules, including authentication, user profile management, student group registration, supervisor dashboard functions, and academic data storage. Authentication ensures that only authorised users can access the relevant features. For example, students can submit research ideas and send supervisor requests, while supervisors can view and respond to requests assigned to them. User profile and academic data are used to provide supervisor details, research interests, availability notes, and workload information needed for the recommendation process.

Integration is mainly achieved through REST API calls and shared database records. The frontend communicates with the backend when students submit research details, view recommendations, or send requests. The backend then coordinates the required operations, such as retrieving profile data, creating request records, updating assignment details, and invoking the recommendation service. The recommendation service is not called directly from the frontend; it is accessed through the backend so that authentication, validation, and role-based access remain controlled by the main application layer. This design keeps the recommendation process connected to the wider workflow while maintaining proper control over user actions and data access.

5.4 Verification and software testing

Verification and testing were carried out to confirm that the component behaves correctly across both the recommendation process and the supervisor request workflow. The first test area was topic processing. In this scenario, a student enters a research title, abstract, and keywords through the Find Supervisor interface. The expected behaviour is that the system extracts relevant topic terms from the submitted research idea and prepares the input for supervisor matching. This was verified by reviewing the recommendation service response and checking whether the user interface displayed recommendation output based on the submitted topic.

The recommendation process was tested by requesting ranked supervisor suggestions for different student research ideas. The expected behaviour was that the system would return supervisor recommendation cards with relevant supervisor information, relevance explanations, workload indicators, and available request actions. The API response and the

frontend output were reviewed to confirm that the returned recommendations were correctly displayed and understandable to the student.

Workload refresh was also tested because supervisor availability can change after assignments are made. In this scenario, changes were made to active supervision assignments, and later recommendation requests were checked to confirm whether workload labels were updated correctly. This was verified by comparing database records with the API output and the workload information displayed on the interface.

The request workflow was tested using primary supervisor and co-supervisor request scenarios. When a student sent a request to a selected supervisor, the expected behaviour was that a pending request would be created and made visible to the relevant supervisor. This was verified by checking the stored request record and the supervisor-facing dashboard. Acceptance and rejection scenarios were then tested from the supervisor side to confirm that supervisor decisions were handled correctly.

Special attention was given to assignment and conflict-handling behaviour. When a supervisor accepted a request, the system was expected to assign the supervisor to the correct role and cancel conflicting pending requests for the same role. This was verified by reviewing the updated database state and checking whether the student and supervisor dashboards reflected the correct assignment status. Role restriction was also tested by attempting to create a new request after a role had already been assigned. The expected behaviour was that the system would block or disable further requests for the same role, preventing duplicate supervisor allocation.

Table 5. Test areas with scenarios

Test area	Scenario	Expected behaviour	Evidence type
Topic processing	Student enters title, abstract, and keywords.	System extracts topic terms and prepares matching input.	Service response and UI output
Recommendation	Student requests ranked supervisors.	System returns supervisor cards with relevance and workload indicators.	API response and UI review
Workload refresh	Assignments change in database.	Workload labels update during later recommendation requests.	Database and API comparison
Primary request	Student requests primary supervisor.	Pending request is created for selected supervisor.	Request record
Acceptance	Supervisor accepts request.	Role is assigned and conflicting pending requests are cancelled.	Database state and dashboard
Role restriction	Role already assigned.	New request for same role is blocked or disabled.	UI state and API response

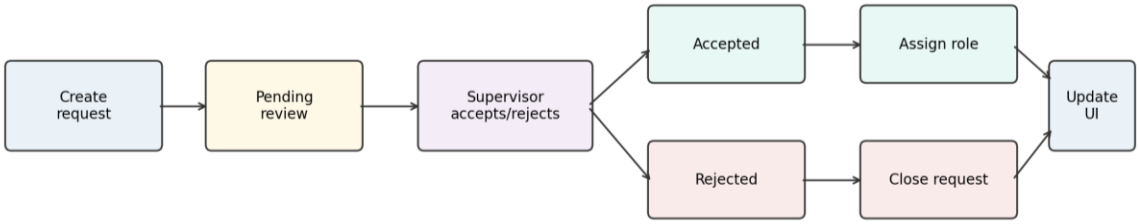


Figure 3. Supervisor Request flow

5.5 Deployment and operation

For deployment, the system can be hosted using Microsoft Azure services. The frontend can be deployed as a web application, while the backend and recommendation service can be deployed as separate application services or containerized services. The database can be hosted using an Azure-supported PostgreSQL service. This deployment approach allows each part of the system to run independently while still communicating through secured API endpoints. It also supports scalability, monitoring, and easier maintenance of the different service layers.

In a production environment, secure environment management should be used to protect database credentials, API keys, service URLs, and other configuration values. Role-based API protection should also be enforced so that students, supervisors, and administrators can only access the operations permitted for their role. Monitored service logs should be maintained for the frontend, backend, recommendation service, and database interactions. These logs are useful for identifying failed recommendation requests, request workflow errors, service downtime, and integration issues during operation.

6. RESULTS AND VALIDATION

6.1 Research results

The research outcome demonstrates that supervisor recommendation can be understood as an integrated academic decision-support process rather than a standalone ranking task. The component showed that supervisor selection requires more than identifying the supervisor with the highest textual similarity to a student's research topic. A suitable supervisor must be academically relevant, connected to the research area, practically available, and willing to accept the supervision responsibility.

The component combines four decision layers: topic understanding, graph-based relationship scoring, workload feasibility, and supervisor approval. Topic understanding supports the identification of supervisors whose expertise is related to the student's title, abstract, and keywords. Graph-based scoring adds an additional layer by considering academic relationships between supervisors, research ideas, keywords, and collaboration links. Workload feasibility ensures that recommendations are not based only on expertise, but also consider current supervision responsibilities. Finally, supervisor approval ensures that the

system remains human-in-the-loop and does not automatically assign supervisors without academic consent.

The evidence for this result comes from the completed workflow scenarios used during validation. In these scenarios, a student submitted a research idea, the system generated ranked supervisor suggestions, workload information was displayed, requests were created for specific roles, and supervisors accepted or rejected those requests. These scenarios showed that the recommendation output could be connected to actual assignment behaviour, supporting the argument that supervisor recommendation should be treated as both an algorithmic and workflow-based problem.

Manual relevance validation was performed using twenty test research topics with five recommendations per topic, producing 100 recommendation judgments. Of these recommendations, 20 were marked as highly relevant, 69 were marked as partially relevant, and 11 were marked as not relevant. Therefore, 89 of the 100 top-five recommendations were considered relevant under the binary relevance definition.

The system achieved 100% Top-1 relevance accuracy because every test topic had a relevant supervisor in the first ranked position. Precision in top 5 results was 89.0%, indicating that most of the top-five recommendations were useful. Hit in top 5 results was 100%, showing that the system always returned at least one relevant supervisor within the top five. MRR was 1.00 because the first relevant result appeared at rank one for every topic. NDCG in top 5 results was 94.1%, showing strong rank quality under graded relevance.

Table 6. Test results

Metric	Result	Interpretation
Top-1 relevance accuracy	100%	The first recommended supervisor was relevant for all twenty topics
Precision	89.0%	Eighty-nine out of one hundred top-five recommendations were relevant
Hit	100%	Each topic had at least one relevant supervisor in the top five
MRR	1.00	The first relevant supervisor appeared at rank one for all topics
NDCG	94.1%	Highly relevant supervisors generally appeared near the top

6.2 Product and Technical Results

The developed prototype supports the main functional requirements of the component by allowing students to submit structured research details, including the research title, abstract, and keywords. These inputs are used to generate supervisor recommendations, which are displayed through recommendation cards containing supervisor information, relevance-related details, workload labels, availability notes, and request options. This provides students with more decision support than a static supervisor profile page or a simple keyword search interface, because the system not only displays supervisor information but also helps students understand suitability and practical availability.

The prototype also supports the workflow required after recommendations are generated. Students can create role-specific requests for primary supervisor or co-supervisor roles, while supervisors can review, accept, or reject those requests. When a request is accepted, the system updates the relevant role assignment and cancels conflicting pending requests for the same role. Technical validation showed that the component can coordinate actions across the frontend, backend, recommendation service, and database under controlled scenarios. Evidence for this includes correct recommendation display, request state updates, assignment changes, workload-related output, API responses, database state changes, and observed user interface behaviour during workflow testing.

6.3 User or Stakeholder Validation

Stakeholder validation was carried out through scenario-based review rather than formal deployment with real institutional users. The validation scenarios were designed to represent realistic interactions between students and supervisors. A typical scenario involved a student entering a proposed research topic, reviewing the recommended supervisors, checking relevance explanations, workload labels and availability notes, sending a role-specific request, and receiving an outcome after the supervisor accepted or rejected the request. This helped assess whether the workflow was understandable, complete, and practical from the student perspective.

The scenarios also reviewed the supervisor-side workflow and role-based behaviour of the component. From the supervisor perspective, the validation checked whether requests could be viewed, assessed, accepted, or rejected clearly. Evidence for this validation was based on functional walkthroughs, interface review, API behaviour, and expected state changes. For example, once a supervisor accepted a request, the student's assignment status was updated and conflicting pending requests for the same role were cancelled or disabled. Similarly, the system prevented students from continuing to send requests for a role that had already been filled. These behaviours indicate that the component follows the practical supervision process, where students initiate requests but final assignment depends on academic approval.

6.4 Integration Results

Integration validation showed that the component depends heavily on reliable communication between the frontend, backend, recommendation service, and database. The recommendation output must be usable by the request workflow, and the request workflow must update assignment records in a consistent way. This means that recommendation generation and supervisor allocation cannot be treated as separate disconnected features.

The validation confirmed the importance of consistent supervisor identification, current workload retrieval, and safe request-state handling. For example, the recommendation service may identify a suitable supervisor, but the backend must still be able to create a valid request for that supervisor. Similarly, when a supervisor accepts a request, the system must update the correct role, cancel conflicting pending requests, and reflect the updated status on both student and supervisor interfaces.

Evidence for integration results was obtained by comparing API responses, database updates, and frontend states after each workflow action. When a student sent a request, the request

record had to appear correctly. When a supervisor accepted the request, the assignment state had to change correctly. When the role was already filled, further conflicting requests had to be blocked or disabled. These observations show that database consistency and API contracts are central to the reliability of the component, rather than secondary implementation details.

6.5 Summary of Findings

The validation indicates that a supervisor recommendation component becomes more useful when it is connected to practical allocation actions. Ranked recommendations help students identify relevant supervisors, but ranking alone is not enough for real academic use. Students also need to understand why a supervisor is suggested, whether the supervisor appears available, and whether a formal request can be made for the required role.

The findings show that workload labels, availability notes, request restrictions, supervisor approval, automatic assignment, and cancellation of conflicting requests make the recommendation process more practical. These features help bridge the gap between identifying a suitable supervisor and completing a valid supervisor allocation. The system therefore supports both decision-making and workflow execution.

Overall, the component demonstrates that supervisor selection should be supported through a combined approach. Topic relevance and graph-based scoring improve the recommendation process, while workload feasibility and request handling make the output usable in a real academic environment. The main finding is that the value of the component lies not only in generating recommendations, but in connecting those recommendations to the actual supervision assignment process.

7. Discussion

7.1 Interpretation of findings

The main interpretation of the findings is that supervisor recommendation should not be treated only as a similarity-ranking problem. Although topic similarity is important, it does not fully represent the practical realities of academic supervision. A supervisor may be highly relevant to a student's research area but may already have a high supervision workload. Similarly, a supervisor may appear available, but still needs to review whether the proposed topic is suitable for their expertise, current responsibilities, and supervision capacity.

The component therefore uses AI-assisted ranking as a decision-support mechanism rather than an automatic assignment mechanism. Topic understanding, graph-based relevance, and workload feasibility help students identify suitable supervisors more effectively, while the request and approval workflow preserves human judgement. This interpretation supports the view that a useful supervisor recommendation system should combine algorithmic support with practical workflow controls, allowing recommendations to guide the process without replacing supervisor review and academic responsibility.

7.2 Comparison with literature baselines and existing solutions

Compared with manual supervisor selection, the proposed component reduces the effort required for students to identify relevant supervisors. In a manual process, students often

depend on personal knowledge, peer suggestions, or limited staff profile information when deciding whom to approach. Prior supervisor recommendation studies also identify this issue and propose automated matching as a way to support students in finding suitable academic staff [1], [2], [4]. This can be difficult, especially when students are unfamiliar with the research interests of all available supervisors. The proposed component improves this process by analysing the student's research idea and producing a ranked list of suitable supervisors, allowing students to make a more informed decision.

Compared with static profile search, the component does more than simply display staff information. Static profile pages require students to manually read, compare, and interpret each supervisor's expertise. Existing content-based supervisor recommendation studies commonly use student research titles, abstracts, keywords, supervisor profiles, or publication records to support this matching process [1], [2], [3]. In contrast, the proposed component uses KeyBERT-based keyword extraction and topic matching to identify relevant research areas from the student's title, abstract, and keywords. It then presents supervisors in a ranked order, supported by relevance scores and explanatory indicators.

Compared with keyword-only matching systems, the proposed component adds graph-based academic relationship scoring. Keyword matching and cosine similarity are useful for direct topic comparison, but they may not capture deeper academic relationships [1], [2], [3]. For example, a supervisor may be suitable because they have supervised similar research topics before, are connected to related research ideas, or collaborate with academics in the same domain. Previous work on academic genealogy, multi-criteria supervisor selection, and context-aware collaborator recommendation shows that supervision history, collaboration, and academic networks can provide useful recommendation signals [5], [6], [7], [8]. By modelling supervisors, research ideas, keywords, and relationships as a graph, the proposed component captures these indirect connections and improves recommendations beyond surface-level keyword overlap.

Compared with recommendation-only prototypes, the proposed component also supports the actual supervisor selection workflow. Many existing systems focus mainly on producing a ranked list or decision-support output [1], [2], [10], [6], [7]. However, in a real academic setting, recommendation alone is not enough. Students must be able to send requests, supervisors must be able to accept or reject them, and the system must update the group's assigned supervisor or co-supervisor accordingly. The proposed component therefore integrates request creation, supervisor review, acceptance, rejection, automatic role assignment, and cancellation of conflicting pending requests.

The component also considers workload-aware behaviour. A supervisor may be highly relevant to a student's topic but may already have too many active supervision responsibilities. Existing work on supervisor allocation highlights the importance of balancing expertise alignment with workload distribution [9]. Multi-criteria supervisor recommendation studies also show that supervisor selection may require factors beyond topic similarity, including academic output, collaboration, and supervision-related indicators [6], [7]. To address this, the proposed component includes workload feasibility in the recommendation process and reflects supervision capacity through the user interface.

Therefore, the main strength of the proposed component is not only the use of a single algorithm. Its value comes from combining multiple techniques into one practical academic workflow. KeyBERT supports topic understanding, graph-based scoring captures academic relationships, workload calculation improves feasibility, and the request workflow connects recommendations to real supervisor allocation. This hybrid design better reflects the real constraints of academic supervision, where expertise, workload, role availability, supervisor acceptance, and student choice must work together.

7.3 Research contribution

The research contribution of this thesis is the development of a combined framework for supervisor recommendation that considers topic relevance, academic relationships, and workload feasibility together. Instead of treating supervisor selection only as a text-matching problem, the component shows how student research ideas can be matched with supervisor expertise while also considering wider academic connections and practical supervision capacity. This contributes to the understanding that academic recommendation systems must support both information retrieval and institutional decision-making requirements.

The thesis also demonstrates how graph-based academic relationship modelling can complement direct text similarity in supervisor recommendation. While topic matching helps identify supervisors with relevant expertise, graph-based scoring adds context by considering relationships between supervisors, research topics, keywords, and collaboration links. Together with workload awareness and supervisor approval, this framework provides a more practical approach to supervisor recommendation, where the goal is not only to rank supervisors but to support a realistic academic allocation process.

7.4 Product contribution

The product contribution of this thesis is a working software component that connects supervisor recommendation with the practical workflow of supervisor allocation. The component allows students to enter their research idea, view recommended supervisors through clear recommendation cards, understand supervisor suitability and workload status, and send role-specific requests for either a supervisor or co-supervisor role. This reduces the effort required for students to manually search for suitable academic staff and gives them more useful information before making a request.

The component also contributes to the supervisor and administrative side of the allocation process by supporting backend request handling, supervisor acceptance and rejection, automatic role assignment, and cancellation of conflicting pending requests. This makes the product more than a recommendation display feature; it becomes a workflow-support tool that helps manage supervisor selection in a controlled and consistent way. By combining student-facing recommendations with supervisor-facing decision actions, the component improves transparency, role clarity, and practical usability in the supervision allocation process.

7.5 Limitations and threats to validity

The evaluation is based on manual relevance judgments rather than long-term historical acceptance data. While this is suitable for validating recommendation relevance during

development, future evaluation should include real student usage, supervisor acceptance patterns, and coordinator feedback across multiple project cycles.

The recommendation quality also depends on the completeness of supervisor profile data and previous research idea data. If a supervisor profile is incomplete, the model may underestimate the supervisor relevance. Further work should include profile update workflows, publication imports, and periodic validation of expertise data.

Finally, workload thresholds may need to be customized for institutional policy. The same number of groups may not represent the same workload for every supervisor because project complexity, student independence, and supervisor role expectations vary.

7.6 Implications

The findings imply that academic recommendation systems should be designed as decision-support workflows rather than isolated ranking engines. In supervisor selection, a ranked list alone is not enough because the final decision depends on topic suitability, supervisor workload, role availability, and academic approval. Therefore, similar systems should combine recommendation logic with practical workflow features such as request handling, supervisor review, assignment updates, and conflict prevention.

For institutions adopting similar systems, the quality of the output will depend heavily on the accuracy of supervisor profiles, research interest data, workload records, and availability information. Clear capacity thresholds should also be defined so that workload labels are meaningful and consistent. Most importantly, final supervisor assignment should remain transparent and reviewable by academic staff, ensuring that AI-assisted recommendations support human decision-making rather than replacing academic judgement.

8. Commercialization and sustainability

8.1 Value proposition and target users

The value proposition of this component is to make supervisor selection more efficient, transparent, and manageable. It reduces the effort students spend searching for suitable supervisors by providing ranked recommendations based on research topic relevance, academic relationships, workload status, and role availability. It also reduces uncertainty by giving students clearer information before they send a request, rather than requiring them to depend only on personal knowledge, peer suggestions, or manual profile browsing.

The target users include undergraduate and postgraduate students, academic supervisors, project coordinators, and academic departments involved in research project allocation. Students benefit by receiving clearer guidance when selecting supervisors, while supervisors benefit from a structured request and review process. Project coordinators and departments benefit because the component supports more organised allocation, improves visibility of supervisor capacity, and helps reduce duplicate or conflicting supervisor assignment requests.

8.2 Market and competitive position

The component is positioned as an academic workflow enhancement rather than a general-purpose recommender product. Its focus is not only to suggest supervisors, but to support the

actual process of supervisor allocation within an academic institution. Unlike simple staff profile search tools or supervisor directories, the component combines research topic matching, workload visibility, availability information, and role-based request handling. This makes it more suitable for departments that need a structured and controlled way to manage supervisor and co-supervisor selection.

Its competitive strength lies in connecting recommendation output with practical academic workflow actions. Students can move from viewing recommended supervisors to sending formal requests, while supervisors can review and respond to those requests. The system also supports workload-aware decision-making and prevents conflicting role assignments. Therefore, the component is positioned as a practical allocation-support tool for educational institutions that require more than basic supervisor discovery or profile browsing.

8.3 Business or sustainability model

A sustainable model for this component can be based on institutional deployment within universities, faculties, or academic departments that manage research project allocation. The system can be configured at department level according to supervision rules, available staff, workload limits, supervisor roles, and programme-specific requirements. It can also be integrated with existing student information systems so that student groups, academic staff profiles, and supervision records can be managed more consistently.

Long-term sustainability depends on keeping the system data and rules up to date. Supervisor profiles, research interests, workload thresholds, availability notes, and assignment records must be maintained regularly to ensure that recommendations remain useful. From a technical perspective, sustainability also requires proper database migration management, service documentation, access control, and maintenance of the frontend, backend, and recommendation service. This ensures that the component can continue to support supervisor allocation as institutional needs change.

8.4 Scalability, intellectual property and compliance

Scalability depends on the system's ability to retrieve and process supervisor profiles, research topic data, graph relationships, workload records, and request states efficiently as the number of students, supervisors, and research projects increases. The service-oriented architecture supports this by allowing the recommendation service to be scaled separately from the main web application when recommendation processing becomes more resource-intensive. Intellectual property considerations include ownership of the developed source code, recommendation logic, configuration rules, synthetic datasets, and any institutional academic data used during deployment. Compliance considerations include protecting student and supervisor information, enforcing role-based access control, managing data retention appropriately, and ensuring that AI-assisted recommendations are used responsibly as decision-support outputs rather than final automated decisions.

8.5 Contribution to the Sustainable Development Goals

The component contributes most directly to **SDG 4: Quality Education** by supporting a more informed, transparent, and efficient process for allocating academic supervisors and co-supervisors. By helping students identify suitable supervisors based on research relevance,

workload feasibility, and role availability, the component can improve access to appropriate academic guidance and reduce uncertainty in the supervision selection process. It also supports SDG 9: Industry, Innovation and Infrastructure by demonstrating how AI-assisted recommendation, graph-based modelling, and digital workflow automation can be responsibly integrated into institutional academic systems to improve educational infrastructure and administrative efficiency.

9. Conclusion and future directions

9.1 Conclusion

This research developed a Recommendation and Collaboration Network for the project. The component addresses the problem of supervisor discovery and request management by combining natural language processing, graph neural network scoring, dynamic workload awareness, supervisor availability notes, and a role-based request workflow.

The system uses KeyBERT to improve topic understanding from student titles, abstracts, and keywords. It uses a heterogeneous research graph and GraphSAGE-based scoring to capture relationships among supervisors, research ideas, keywords, and collaborations. It applies workload feasibility as a practical adjustment to ranking and displays workload labels and availability notes for transparency.

The request workflow converts recommendations into actionable supervisor selection. Students can send role-specific requests, supervisors can review the submitted topic and group details, and accepted requests directly assign the supervisor or co-supervisor to the research group. Competing pending requests for the same role are automatically cancelled, preventing inconsistent assignment outcomes.

Evaluation results indicate that the component can produce relevant recommendations, with 100% Top-1 relevance accuracy and 89.0% Precision across the validation set. Functional testing confirmed that the integrated workflow supports recommendation display, request creation, supervisor acceptance, assignment, cancellation, availability notes, and role-filled guards.

9.2 Achievements of objectives

The objectives of the component were achieved through the design and implementation of a complete supervisor recommendation and assignment workflow. The component supports structured student research input, KeyBERT-based keyword extraction, topic similarity matching, graph-based relevance scoring, dynamic workload feasibility, and role-specific request handling. It also supports supervisor decision-making through acceptance and rejection actions, automatic assignment after approval, and cancellation of conflicting pending requests. Scenario-based validation showed that the component can connect algorithmic recommendation with the practical workflow required for academic supervisor and co-supervisor allocation.

9.3 Individual research and product contribution

The individual research contribution of this thesis is the formulation of supervisor recommendation as a socio-technical problem that involves more than ranking supervisors by topic similarity. The component demonstrates that effective supervisor selection must consider research relevance, academic relationships, workload feasibility, role availability, and human acceptance. The product contribution is a modular working component that supports the student and supervisor journey from research idea submission to recommendation, request creation, supervisor review, and final role-specific assignment outcome.

9.4 Future directions

Future work should evaluate the system in a live academic setting using real student interactions and supervisor acceptance outcomes. This would allow the recommendation model to be assessed not only on manual relevance but also on acceptance rate, student satisfaction, supervisor satisfaction, and allocation fairness.

The graph can also be expanded with publication metadata, citation networks, supervisor-student collaboration history, and research lab affiliations. These additions would make the collaboration network richer and may improve the GNN score. Another improvement is to provide more detailed explanations, such as showing which keywords, previous topics, or graph paths contributed most to a recommendation.

Workload modelling can be refined by considering project complexity, supervisor preferences, maximum role-specific limits, and academic calendar events. Finally, periodic retraining or incremental graph updates can ensure that the recommendation service remains current as new research groups, topics, and supervisors are added to the platform.

References

[1] Ridwan Rismanto; Arie Rachmad Syulistyo; Bebby Pramudya Citra Agusta, "Research Supervisor Recommendation System Based on Topic Conformity," 2020.

[2] Zulfa Fajrul Falah; Fajar Suryawan, "Recommendation System to Propose Final Project Supervisor using Cosine Similarity Matrix," 2022.

[3] Maresha Caroline Wijanto; Rachmi Rachmadiany; Oscar Karnalim, "Thesis Supervisor Recommendation with Representative Content and Information Retrieval," *Journal of Information Systems Engineering and Business Intelligence,* 2020.

[4] Gintare Zemaityte; Kasim Terzić, "Supervisor Recommendation Tool for Computer Science Projects," in *Proceedings of the 3rd Conference on Computing Education Practice (CEP ’19)*, 2019.

[5] Gabriel Madeira; Eduardo N. Borges; Giancarlo Lucca; Washington Carvalho-Segundo; Jonata C. Wieczynski; Helida Santos; Graçaliz Dimuro, "Using Academic Genealogy for Recommending Supervisors," in *Proceedings of the 23rd International Conference on Enterprise Information Systems (ICEIS 2021)*, 2021.

[6] Mir Anamul Hasan; Daniel G. Schwartz, "RecAdvisor: Criteria-based Ph.D. Supervisor Recommendation," in *Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2018.

[7] M. A. Hasan, "A Multi-Criteria Decision Support System for Ph.D. Supervisor Selection: A Hybrid Approach," 2019.

[8] Zheng Liu; Xing Xie; Lei Chen, "Context-aware Academic Collaborator Recommendation," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD ’18)*, 2018.

[9] Nike Maulani; Geovanne Farell; Ahmaddul Hadi; Dedy Irfan; Mazen Alzyoud; Sharshova Regina Nikolaevna, "Automated Academic Supervisor Allocation Using the C4.5 Decision Tree Algorithm: A Scalable Web-Based Solution," *Journal of Hypermedia & Technology-Enhanced Learning*, 2025.

[10] Aisyah Fitri; Desty Rodiah; Yunita, "Research Supervisor Recommendation System Using a Hybrid Filtering Approach," in *2025 International Conference on Artificial Intelligence and Technological Solutions (ICAITech)*, 2025.

[11] Dasri; Annisa; Toto Haryanto, "Two-Way Thesis Supervisor Recommendation System Using MapReduce K-Skyband View Queries," *International Journal on Informatics Visualization (JOIV),* 2025.

[12] Hyeyoung Ko; Suyeon Lee; Yoonseo Park; Anna Choi, "A Survey of Recommendation Systems: Recommendation Models, Techniques, and Application Fields," 2022.

[13] "Find a supervisor," [Online]. Available: https://www.auckland.ac.nz/en/research/postgraduate-research-and-study/find-a-supervisor.html.

[14] "Find a Supervisor/PhD Project," [Online]. Available: https://www.universityofgalway.ie/findasupervisor/.

[15] "Supervisors in Information Technology, Monash University," [Online]. Available: https://supervisorconnect.it.monash.edu/supervisors.

[16] "PhD Supervisors," [Online]. Available: https://www.findaphd.com/supervisors/.

[17] Sadura Priscilla Akinrinwa; Samuel Tolulope Ogunjo; Kehinde Dennis Akinrinwa, "NLP Model for Postgraduate Research Examiner Recommendation Based on Scholarly Activity," in *2024 IEEE 5th International Conference on Electro-Computing Technologies for Humanity (NIGERCON)*, 2024.

[18] "Research Supervisor Recommendation System Based on Topic Conformity," [Online]. Available: https://www.mecs-press.org/ijmecs/ijmecs-v12-n1/v12n1-4.html.

[19] Falah, "Recommendation System to Propose Final Project Supervisors using Cosine Similarity Matrix," [Online]. Available: https://journals.ums.ac.id/khif/article/view/16235.

GLOSSARY

GraphSAGE - A graph neural network approach that learns node representations by aggregating information from neighbouring nodes.

Heterogeneous graph - A graph containing multiple types of nodes or edges, such as supervisors, research ideas, keywords, and collaboration links.

KeyBERT - A keyword extraction technique that uses BERT-based embeddings to identify important words and phrases from text.

Supervisor recommendation - The process of suggesting suitable academic supervisors based on topic relevance, expertise, relationship data, and practical constraints.

Workload feasibility - A practical measure of whether a supervisor has capacity to take additional primary or co-supervision responsibilities.

APPENDICES

Appendix A Research instruments or extended results

Test case 1 – results output

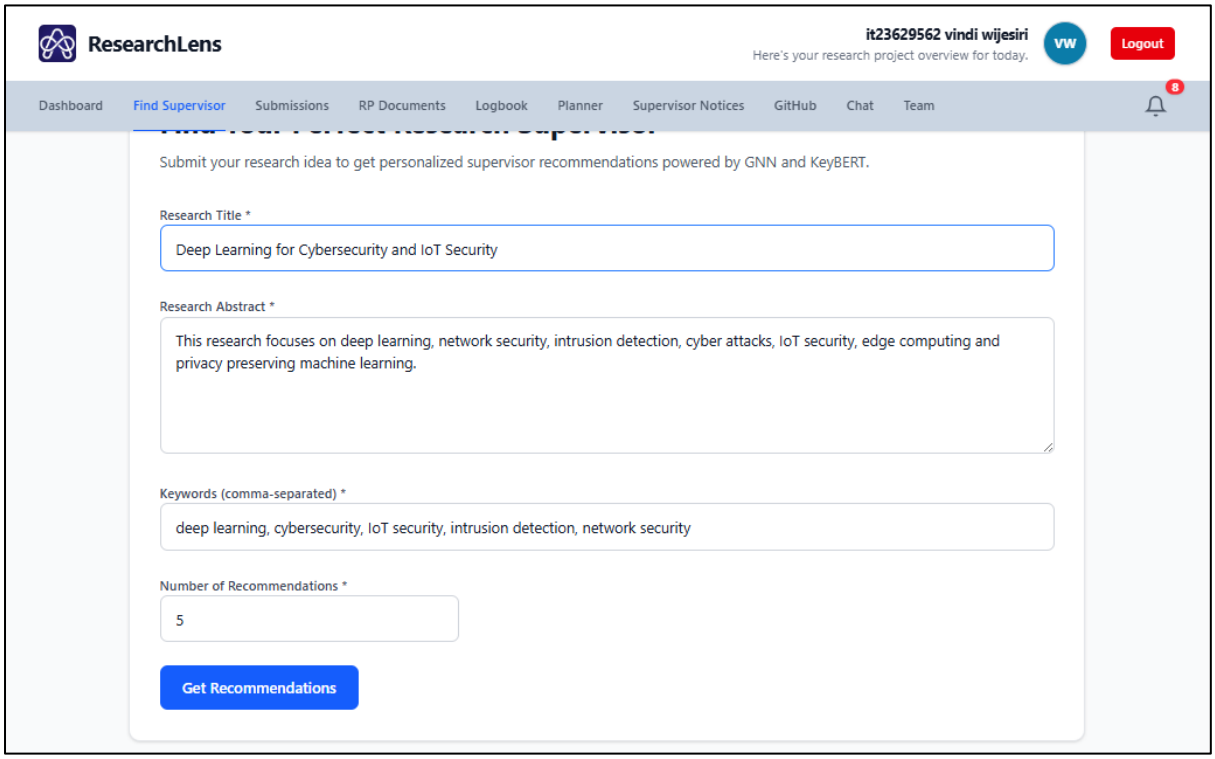


Figure 4. Results input

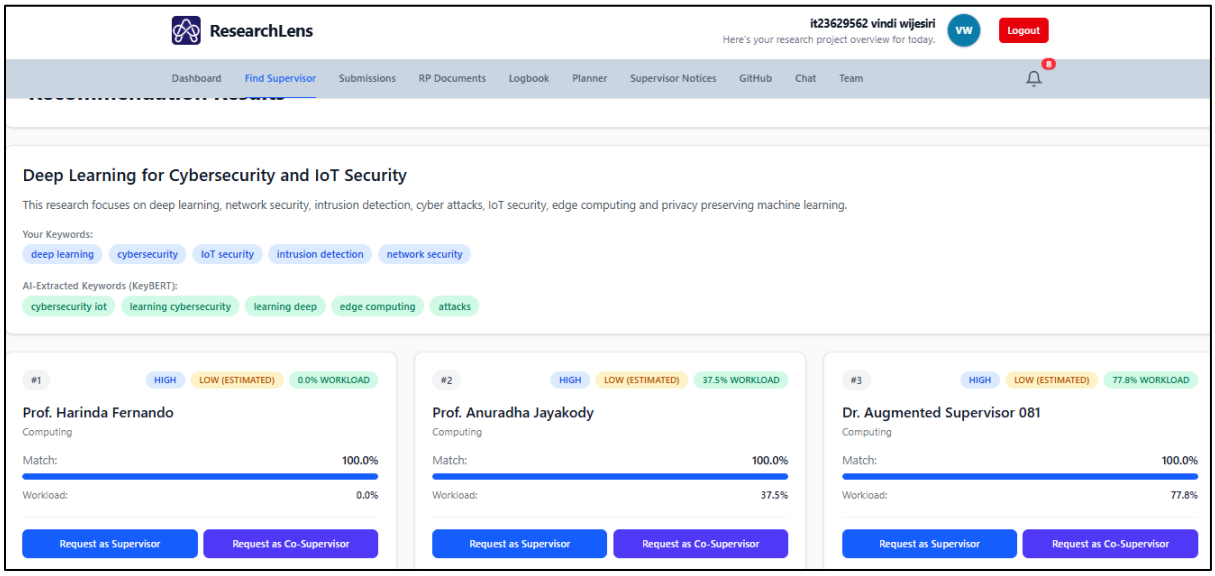


Figure 5. Results output

Test case 2 – results from backend similar to the frontend

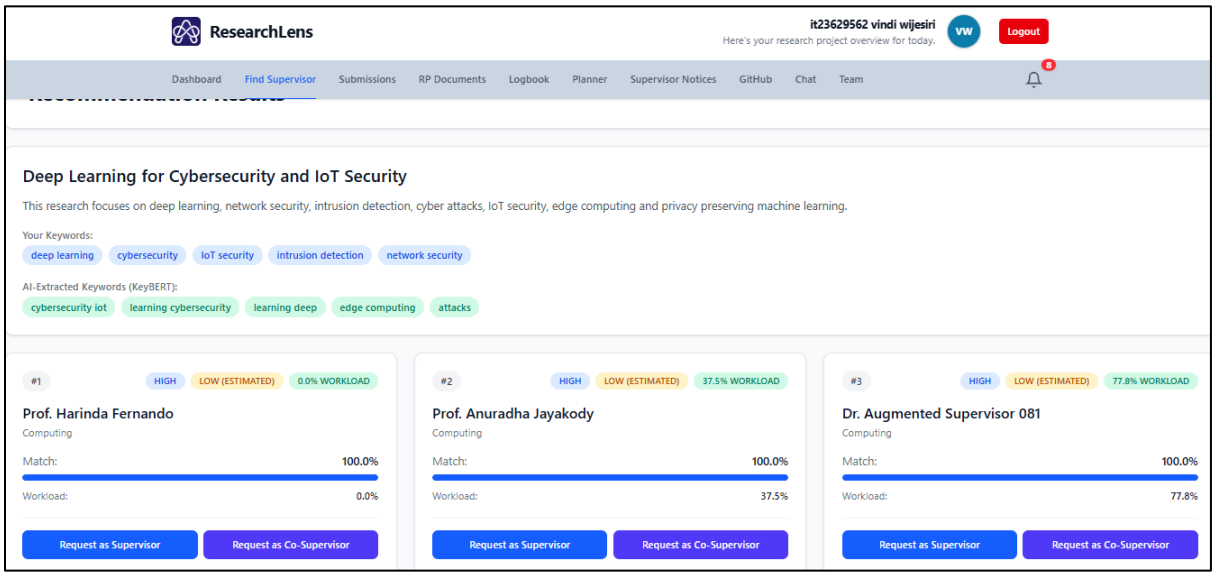


Figure 6. results from frontend 1

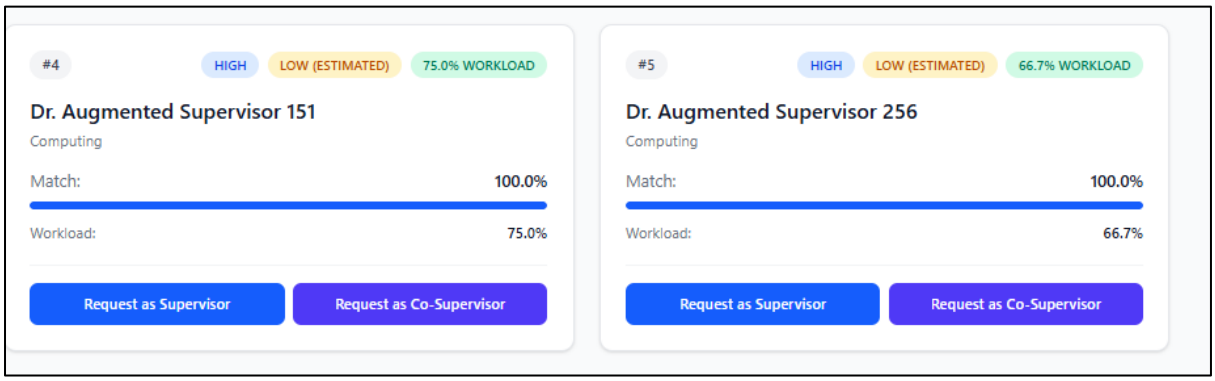


Figure 7. Results from frontend 2



Figure 8. Results from backend 1

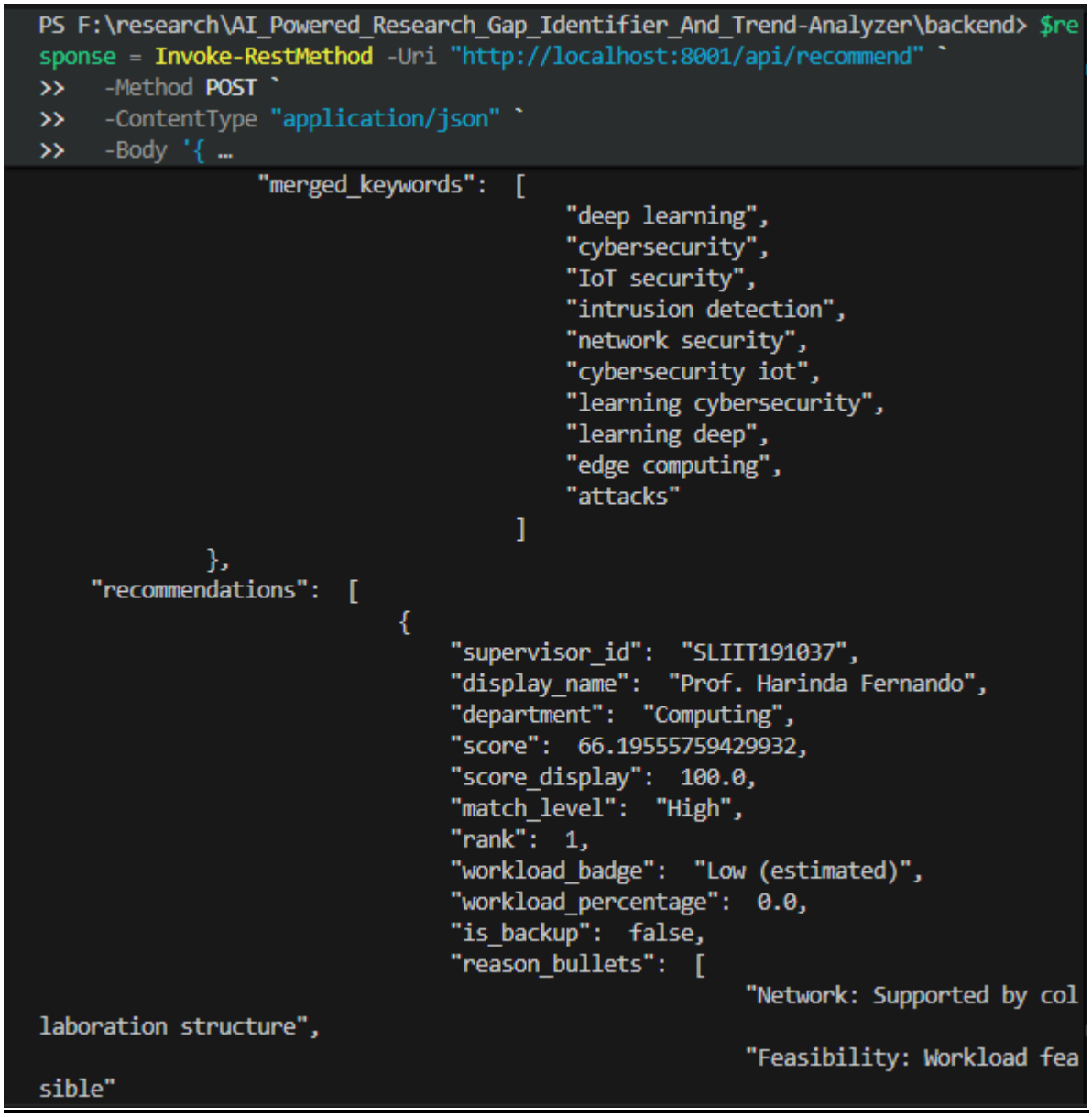


Figure 9. Results from backend 2

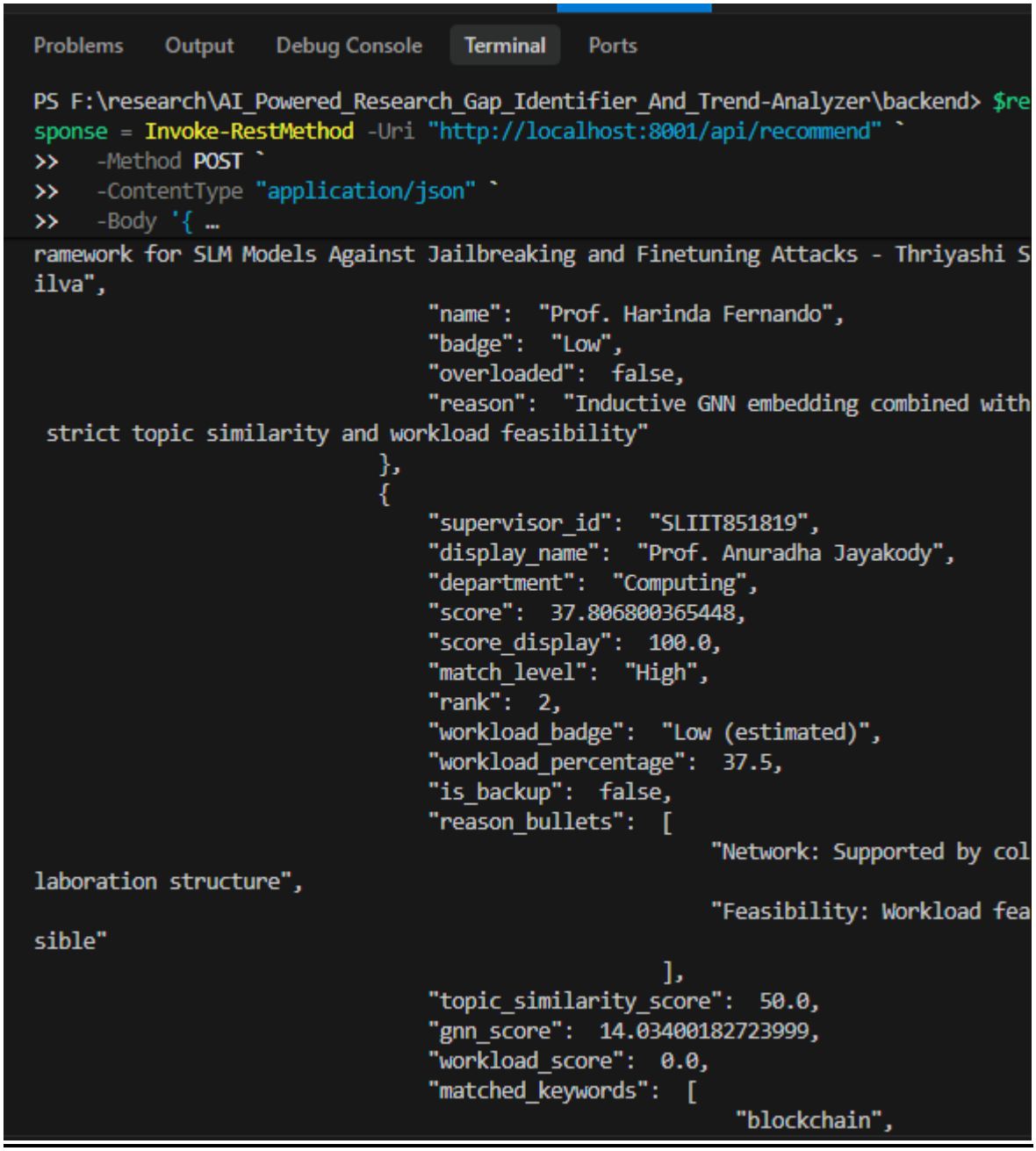


Figure 10. Results from backend 3

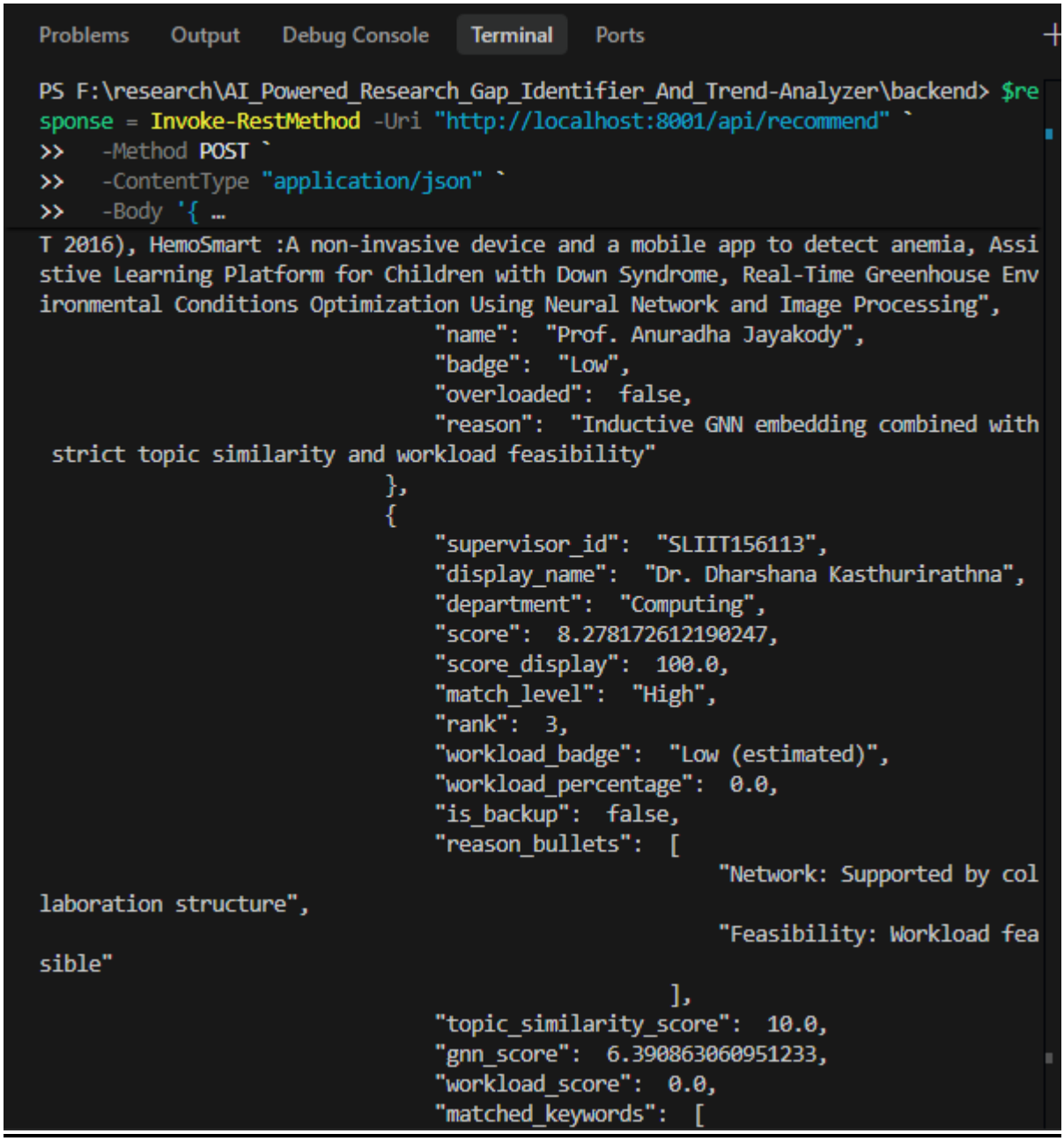


Figure 11. Results from backend 4



```
{
    "supervisor_id": "SLIIT-AUG-0081",
    "display_name": "Dr. Augmented Supervisor 081",
    "department": "Computing",
    "score": 6.553278684616089,
    "score_display": 99.9,
    "match_level": "High",
    "rank": 4,
    "workload_badge": "Low (estimated)",
    "workload_percentage": 77.8,
    "is_backup": false,
    "reason_bullets": [
                        "Network: Supported by col
laboration structure",
                        "Feasibility: Workload fea
sible"
                    ],
    "topic_similarity_score": 10.0,
    "gnn_score": -2.2336065769195557,
    "workload_score": 0.0,
    "matched_keywords": [
                            "information security"
                        ],
    "matched_keyword_count": 1,
```


Figure 12. Results from backend 5



```
{
    "supervisor_id": "SLIIT-AUG-0151",
    "display_name": "Dr. Augmented Supervisor 151",
    "department": "Computing",
    "score": 5.599922776222229,
    "score_display": 99.6,
    "match_level": "High",
    "rank": 5,
    "workload_badge": "Low (estimated)",
    "workload_percentage": 75.0,
    "is_backup": false,
    "reason_bullets": [
                        "Network: Supported by col
laboration structure",
                        "Feasibility: Workload fea
sible"
                    ],
    "topic_similarity_score": 10.0,
    "gnn_score": -7.000386118888855,
    "workload_score": 0.0,
    "matched_keywords": [
                            "information security"
```


Figure 13. Results from backend 6

Table 7. Frontend and backend results

Frontend results	**Backend results**
Prof. Harinda Fernando	Prof. Harinda Fernando

Prof. Anuradha Jayakody	Prof. Anuradha Jayakody
Dr. Aug sup 081	Dr. Dharshana Kasthurirathna
Dr. Aug sup 151	Dr. Augmented Supervisor 081
Dr. Aug sup 256	Dr. Augmented Supervisor 151

Test 3 – duplication prevention

After a supervisor accepting, the student can't resend request to the same supervisor it will be displayed as already accepted.

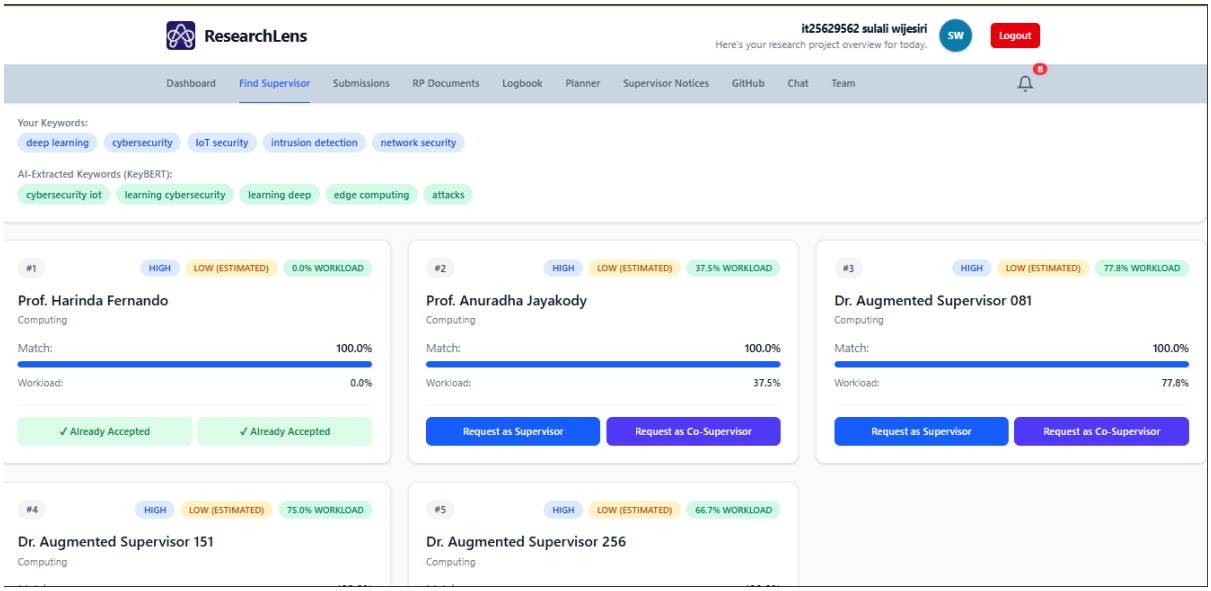


Figure 14. Duplication Prevention

Appendix B Academic integrity and AI use disclosure

Generative AI assistance was used for limited support purposes, including paragraph refinement, language improvement, structure planning, and image generation for thesis-related visual materials. ChatGPT was used to improve the clarity, flow, grammar, and professional tone of selected draft paragraphs, while preserving the original research direction, system design, implementation details, and individual contribution.

Generative AI was also used to assist with creating or refining visual materials used to explain the component, such as architecture, workflow, and conceptual diagrams. The generated content was reviewed, edited, and verified by the author before inclusion. The final responsibility for the accuracy of the thesis content, technical explanations, citations, implementation details, results, and conclusions remains with the author. No AI tool was used as a substitute for the actual system implementation, testing, or academic judgement.